University of California, Irvine

Computer Vision: Creating a 3D Model from 2D Scans

The Statue of David

Kyler Narvaez

Compsci 117: Project in Computer Vision

Professor Charless Fowlkes

3 June, 2026

Project Overview

Goal: Given some scan data, which will be described later, apply a multitude of computer vision techniques to have a finished 3d model that can be used in different softwares, such as Maya or Blender. For the purposes of this report the object in question is a ceramic garden gnome named David.

The program that I created follows this general structure: Calibrate the cameras, find the extrinsic parameter of the cameras, reconstruct the 3d point clouds of the space seen, obtain the meshes created from such point clouds, and align each of the completed meshes onto each other. The rest of the structure occurs via Meshlab where there is smoothing, poisson reconstruction, and other cleaning applications of the final mesh.

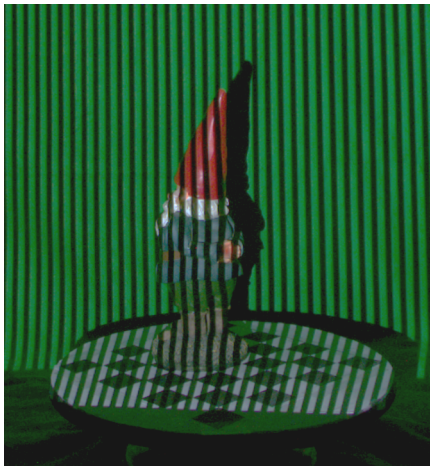
Data

There were two folders of data given in order to start the project. First, was a calibration file which had twelve 1920×1200 pixel images of a large checkerboard positioned in different ways. This would be used to find the intrinsic values of the two cameras used.

Second, there was the “david” folder, which contained 5 “grab” folders. Each grab folder represented one instance of the object, where we would see the gnome from a left and right camera. The context of the scene is our main object on top of a turntable with a green screen background behind it. This structure allowed for the cameras to stay stationary, while the object rotated for each scan. For example, grab_0 was front facing, grab_1 was rotated clockwise, and grab_2 was back-facing. Each grab folder contained the content below:

- Two left camera colored images (color_C1_*)
- Two right camera colored images (color_C0_*)
- Forty left camera structured illumination scans (frame_C1_*)
- Forty right camera structured illustration scans (frame_C0_*)

Essentially each camera had a background camera image, where the object was not yet added to the turntable, and a regular color image, which was just of the normal scene. This will be later used for background subtraction. Note: Due to the setup of the scene, grab_0 is the only folder that has the background image, since the cameras do not move and instead the background is the same for all other color images.



Each camera also used structured illumination, which is where a projector adds binary images (horizontal/vertical black and white blocks) onto the scene. Frames 0-19 where vertical lines and 20-39 were for horizontal lines. The images above (edited for ease of viewing) show grab_2's frame_C0_07 and frame_C0_14, as you may tell the amount of lines doubled per iteration, but they also flip, so that frame0 has black/white and frame1 has white/black. This data will be used later for triangulation, more specifically the decoding of each scan.

Algorithms

There are three types of functions that were present within the pipeline. Python files provided by the professor, past class assignment functions that I had previously worked on, and finally new functions implemented myself in order to adhere to the structure of the data and other goals. Some unmentioned functions, such as build_meshes and apply_reconstructions are just house keeping functions that apply the actual functions over all the scans.

Given Python Files

Calibrate: Used along with the calib data folder to find the corners of the checkerboard and then find the intrinsic parameters, such as focal length, principal point, distance, and radial distortion.

Camutils: A function holder of past created assignment functions that would be unedited for the pipeline. Functions such as makerotation(), triangulate(), residuals(), calibratePose(), and the camera class containing methods project() and update_extrinsics(). Essentially, it is a building block for going from 2d scans to 3d point clouds, which I will describe with my reconstruction function.

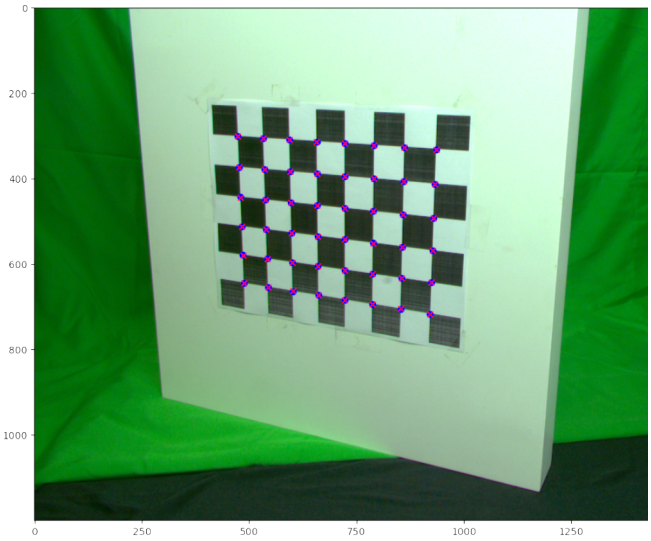
Meshutils: Convert the 3d points, vertex colors, and the mesh triangles into a .ply file that you could name. Was used to convert the meshes automatically and add them to a folder.

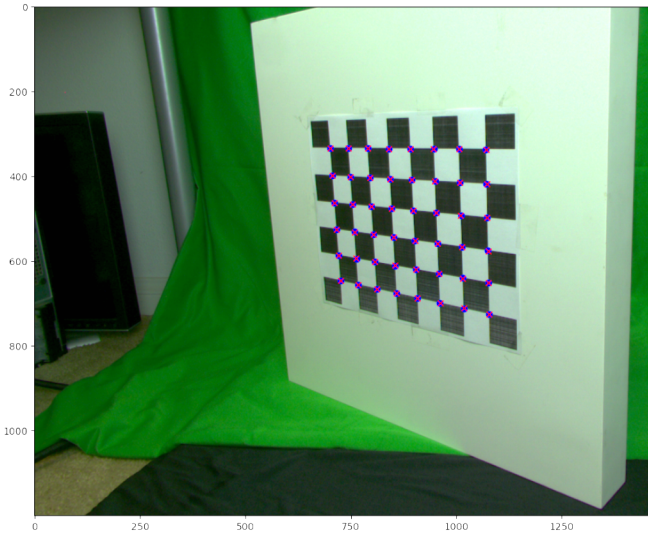
Visutils: Visualization helper that made nice looking graphs for point clouds and meshes. Essentially, just put cameras, points, and meshes in a world space that allowed for trouble shooting.

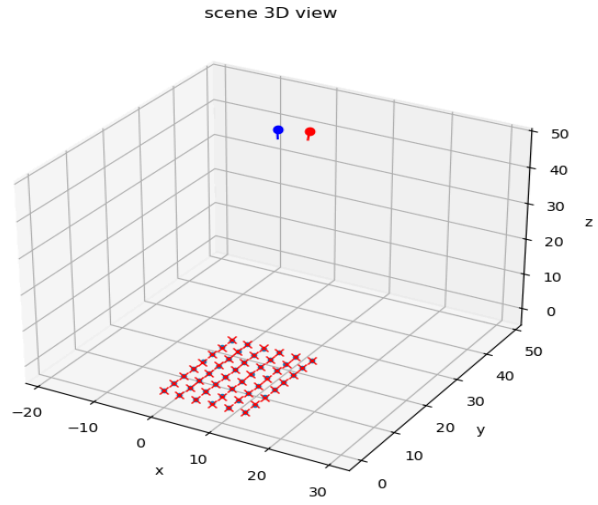
Pipeline Functions

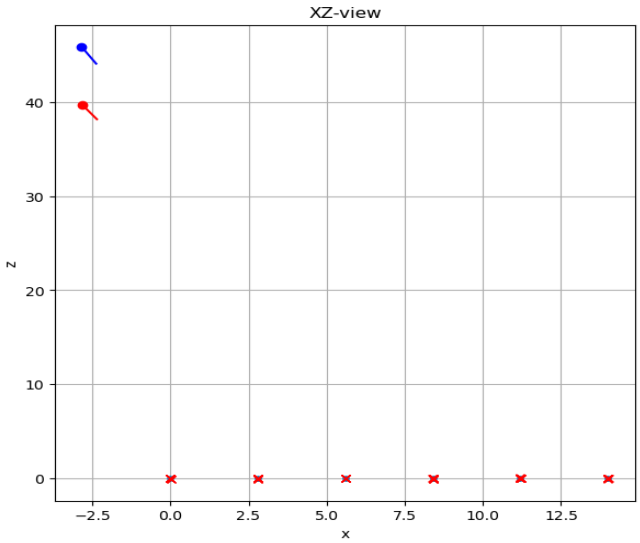
Calibrate_camera: This is the official first step of the pipeline where this function grabs the intrinsic values from the calibrate.py, creates camera objects of them, and then using a single calibration image (checkerboard) applies the calibratePose function on both cameras, which is given initial extrinsic parameters in order to optimize the rotation and translation of the camera until the error between the corner points and the function's projected points are minimized. In summary, find the extrinsic parameters of both cameras.

Results after using the initial parameters of [-45, 0, 0, 0, 0, -50] and [0, 0, 0, 0, 0, -100] for the left and right cameras respectively. Below are the checkerboard corners being correctly mapped by the cameras and a 3d view of the scene.









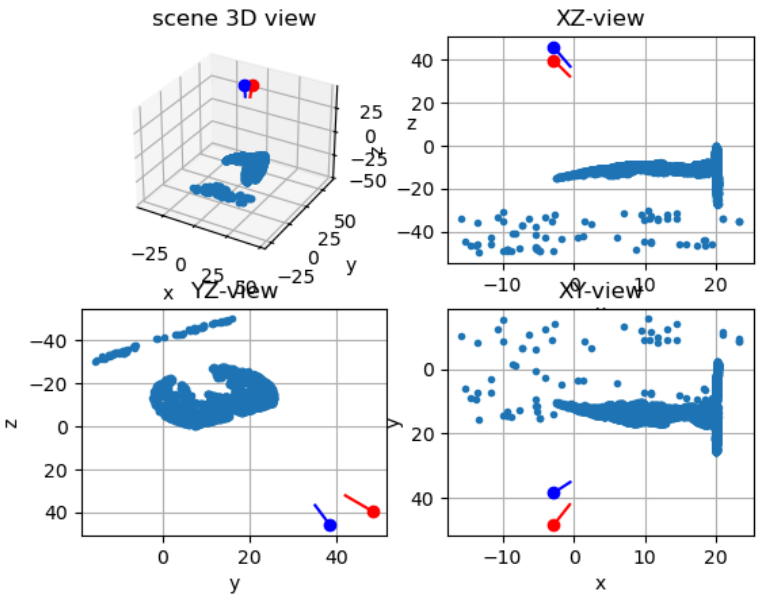
Reconstruction: This function's main goal is to obtain the 3d points of each scan. It does this mainly through structured illumination. Since each point has a binary representation with the earlier described data, we can determine whether two points are the same if they have the same binary coding. These encodings are computed by the “decode” function described below in both the horizontal (frames 0-19) and vertical (frames 20-39) directions. After obtaining the encodings, we can match encodings to points, only keeping points that have valid encodings (seen in both cameras and only one match per point). We can then take all these 2d points and

triangulate them in order to get the 3d point cloud of the scene. Note: This function was edited to include a method of reducing noise through background subtraction (explained below) and also kept track of the color of each point in order to map to the mesh later.

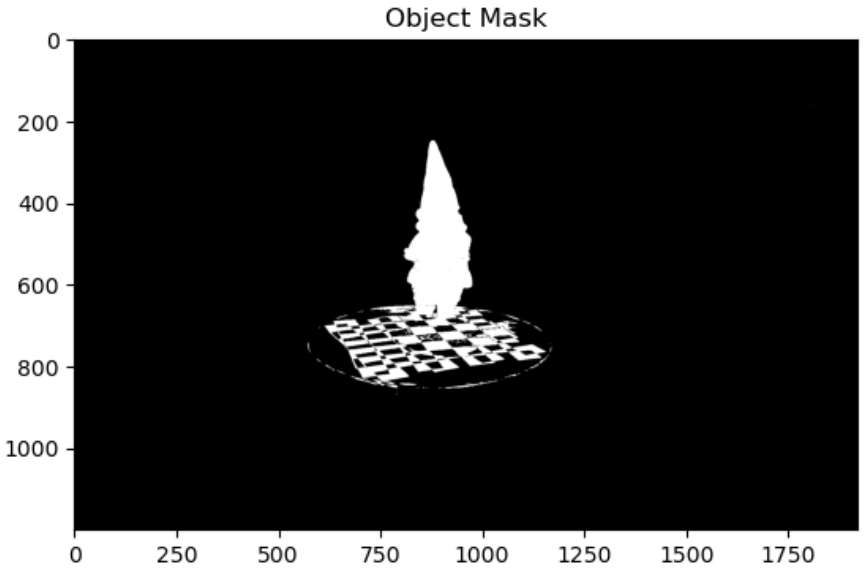
Decode: This function is meant to take in 20 of the frame data and give each point a 10 bit binary representation. It takes every consecutive pair of frames (0-1, 2-3, 4-5, etc) as those pairs are the same number of cuts, but in reverse order. We then set the bit if the first image is greater than the second. However, this is the gray code representation, which is used so that two nearby points do not have drastically different values in bit-representation. Therefore, we convert the 10 bit gray code into binary with a sequence of XORs. The last important application of this function is determining the decodability of each point, which essentially is "how sure are we that this bit can be decoded?". This is determined if the difference between the first image and the second are above a certain threshold. The threshold used for David was 0.01. If a point is not, then that point will be removed from the triangulation.

Background_subtraction: A fairly simple function that takes in the background image and the current color image of the object. A binary mask the same size of the images is built, setting a point to be "the object" if the color difference is above a certain threshold, 0.08 in this case. This mask is then returned to reconstruction to clean the background noise with the decoded values.

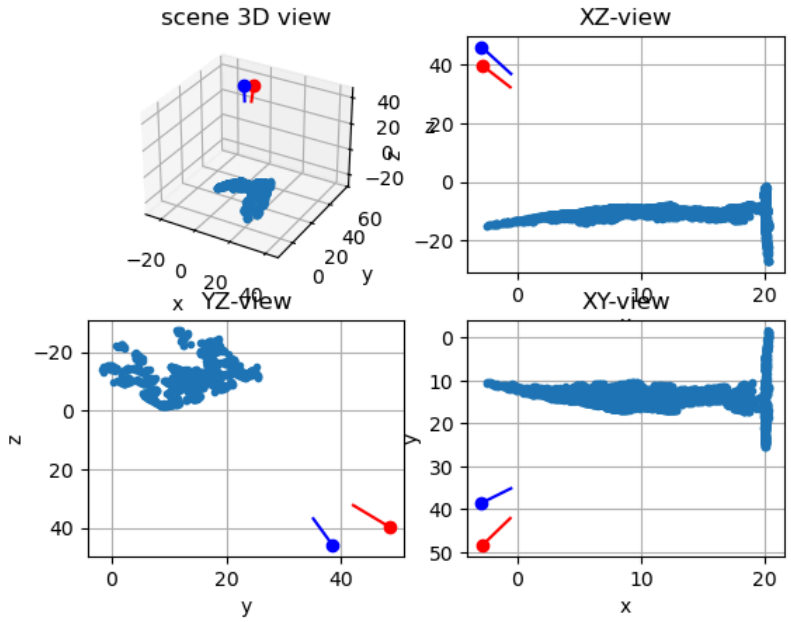
Reconstruction without Background Subtraction



Background Subtraction



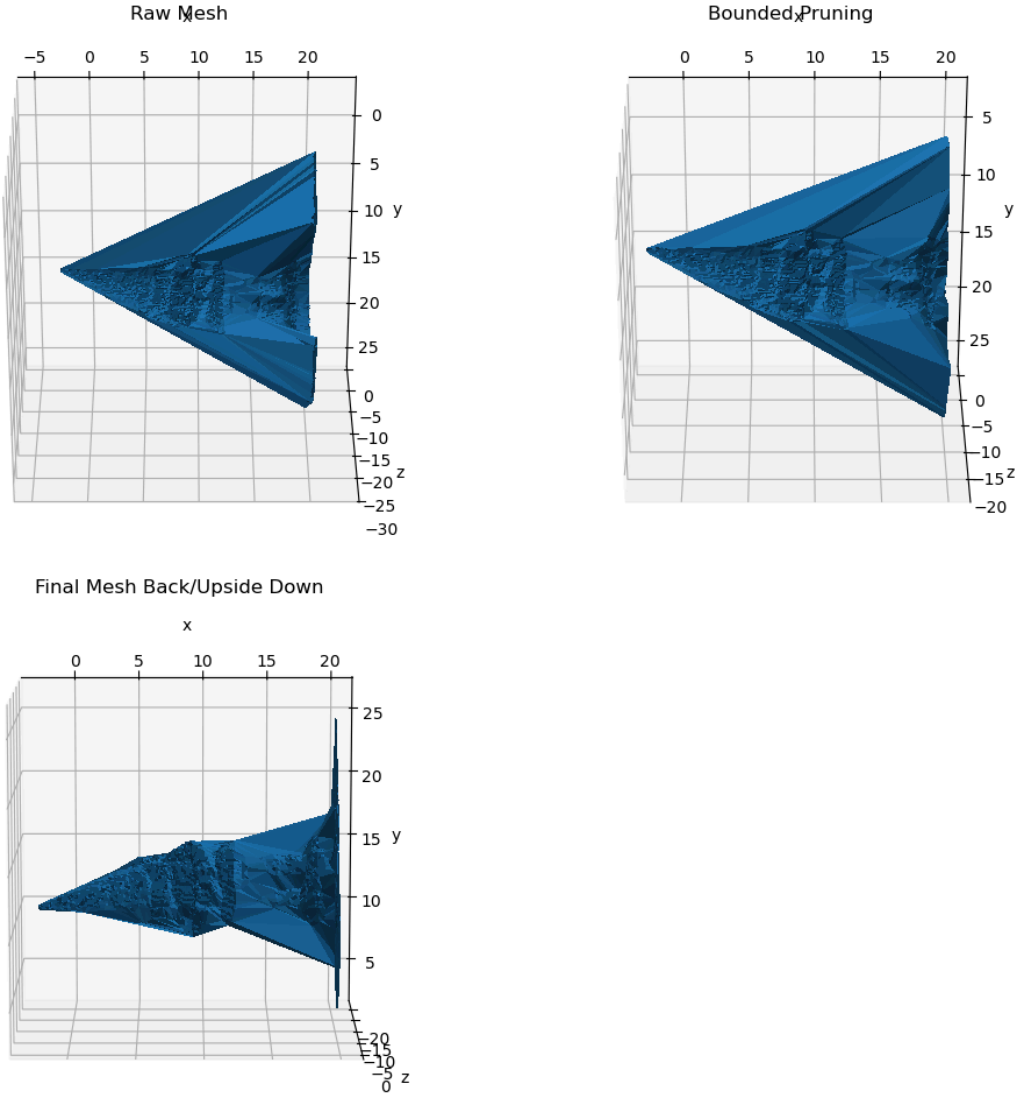
Reconstruction with Object Mask



As you can see background subtraction significantly reduced the messy points behind the triangulated point cloud.

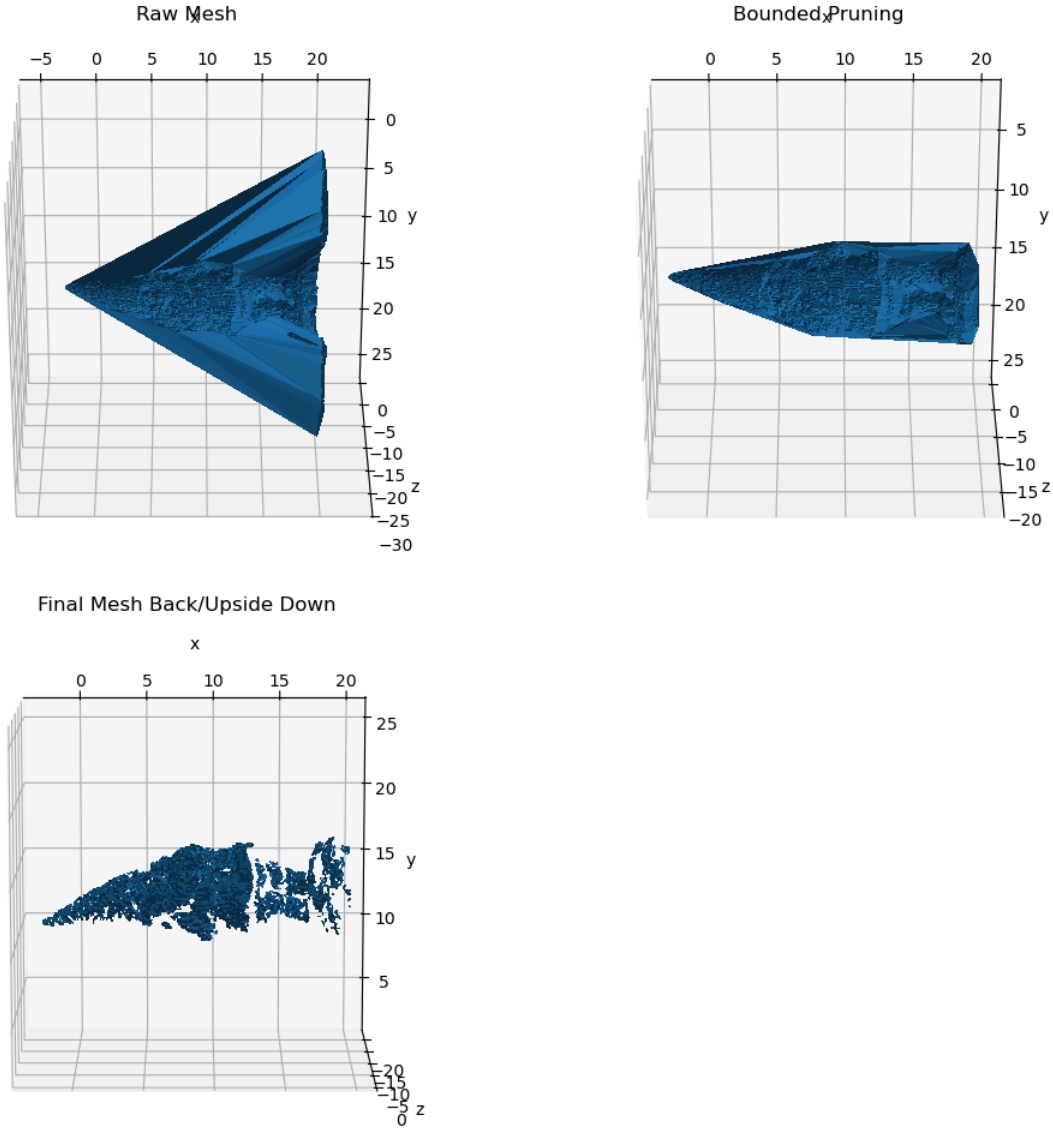
Build_mesh: Responsible for creating triangles that connect between 3d points from reconstruction. The main idea is to use the Delaunay function, which creates non-overlapping triangles, so that no points exist within a triangle. This mostly works well, however cleaning must be done in order to get a better looking mesh. This was implemented in two ways, bounded box limits and a triangle length threshold. The box limits are basically a user defined 3d box that will remove any vertices that exist outside of the box, the most useful application of this was to remove the turntable from the final mesh, which can be seen below as something needed to be fixed.

Bad Boxlimit Parameters



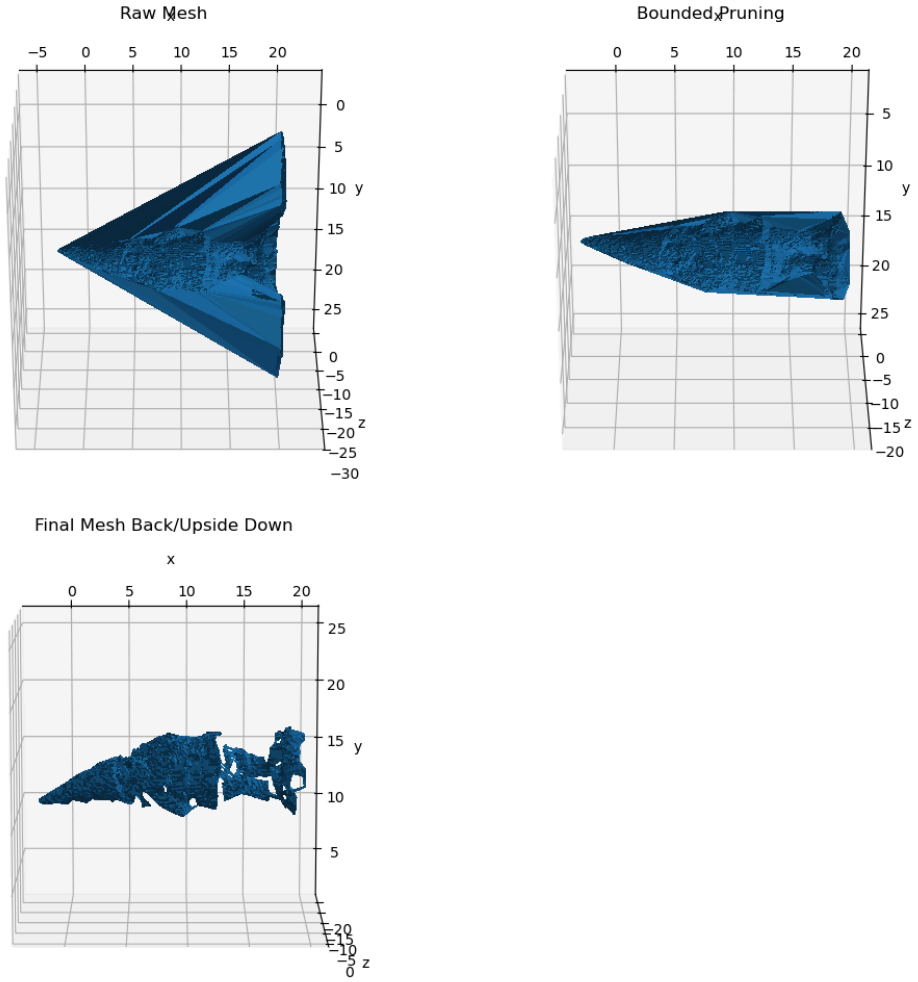
The other mesh cleaning approach was to only allow triangles where all sides were less than a certain threshold where vertices with no triangles would be removed. This was extremely useful, however it would cause too many holes if the threshold was overly aggressive

Bad Triangle Threshold



After experimenting with different parameters, I found that a boxlimit of [-10, 19.5, 8, 18, -20, -8] and triangle threshold of 1.0 obtained good results:

Good Mesh Parameters



Select_corresponding_points: In order to implement alignment there needs to be a way to determine whether two points are the same between scans. The way that it was implemented for this project was user input, where each color image of the 5 scans would allow for the user to click 6-8 points on successive images. This created a correspondence list of points that mapped one scan to another, such as scans 0→1, 1→2, etc. These user clicks would then find the closest 3d point on the point cloud.

Get_alignment: Takes a pair of corresponding points, obtains their center of mass, and applies an SVD approach described in the lecture notes. This computes the rotation and translation needed to get one corresponding point to another.

Align_meshes: We want all the meshes to be aligned to the first scan, however not all scans have matching points with the base scan. Therefore, instead of mapping scanN to scan0, we map scanN to scanN-1, until it makes its way to the base. The helper function, get_all_alignments, creates a list of (R,t) tuples to get from one scan to another and these sets of transformations get applied to their respective scans. For example, scan3 would need to apply $(R,t)_2$ in order to get to scan2, then it would need to apply $(R,t)_1$ to get to scan1, and lastly $(R,t)_0$ to get to scan0. This mostly worked, however the miniscule errors between user clicks compounds for every transformation pair, therefore causing the last mesh to be misaligned fairly noticeable to the base. That meant that the points chosen must be as close to perfect as possible to get good results.

Bad Correspondence Clicks

Results

While experimenting with MeshLab I applied a diverse set of filters. The first filter was to remove any messy or duplicate points. Such as duplicate vertices, vertices that are too close to each other, and parts of the mesh that were disconnected from the main object in order to have a cleaned up mesh.

I then experimented with the Laplacian smoothing technique, where each vertex looks at its nearest neighbors to average out its position.

Laplacian Smoothing with iterations 0, 3, 5, and 10

In order to see which gives the best result I computed the poisson reconstruction on the different smoothed meshes. Below show the resulting meshes:

Poisson Reconstruction on different smoothing

After some consideration I decided to go with the iteration 5 smoothing, as it seemed to keep a majority of the features. So, after cleaning up the mesh some more by removing unused vertices I rendered a final result:

Assessment

This problem was considerably hard. The level of detail needed to create a mesh with specific features such as the face and hands was not receptive with the data and time constraints.

This pipeline was able to clean up the data nicely with background subtraction and decoding the points between the two cameras. Though, I believe that it is the pipeline's mesh creation that is lacking. While using the triangle length threshold it was difficult to find values that would not over mesh, causing bad results with alignment, and under meshing creating too many holes. A better approach would probably be to add vertices to removed triangles to keep some of the detail or some sort of local constraint so that higher frequency vertices would keep their detail.

Another issue was the alignment of the meshes. This problem was extremely difficult to solve because of the compounding error that would occur in the later meshes while applying the transformations. After clicking points more than a hundred times I realized that some points that were being clicked did not exist within the point cloud, causing the nearest point to be drastically different between scans. If this were to be a pipeline that were more user friendly it would be far better to apply automatic correspondence with edge detection. Realizing this I was able to add the point cloud to the click detector, which improved the accuracy of the correspondences, which was the main reason that the final mesh looked well put together.

Evaluation

Overall, I believe that the final mesh matched the original object fairly well. The largest issue was the loss of little details such as the gnome's eyes, teeth, and fingers. These minor details were not able to be caught due to the structured illumination decodability, but also with alignment overlapping certain aspects of the mesh. For example, one mesh could have had the eyes of the gnome, but was overlapped slightly by a mesh that did not have the eyes. Looking closer some smoothing errors can be seen, where the rock on the right side of the gnome is merged slightly with the foot, the belt buckle is not clearly defined, and there are sections that point out oddly like the left side of the rock having a small aberration. If I had more time I believe that the final model could have been smoothed more as the original gnome was extremely smooth in locations such as its hat and coat.

However, I would say that the final model was able to maintain the larger features of the object. For example, the coat keeps its wavy shape in some areas and the pointy nose, cheeks, and ears all stay consistent in the final mesh.

Appendix

House-Keeping Functions: These are just some easy functions that I made to keep the code simple and reusable. Others are for interfacing with the data to apply other functions to all grabs.

save_pickle_data()	open_pickle_data()
convert_to_float()	convert_meshes_to_ply()
apply_reconstructions()	build_meshes()
create_3d_model()	

Past Assignment Functions (No/Slight Changes): These are functions that were made during the assignments from this class that were just plugged in or have small changes such as adding color, masks, or two different cameras.

decode()	reconstruct()	build_mesh()	calibrate_camera()

Functions made from Scratch: These are functions that I made specifically for this pipeline. The get_alignment uses the pseudocode from the slides.

select_corresponding_points()	select_all_corresponding_points()
convert_clicks_to_3d()	get_alignment()
get_all_alignments()	align_meshes()
background_subtraction()	